

《数据库系统原理》报告

数据库系统原理作业 8

张鲔泮

北京交通大学计算机科学与技术学院 AI2201 班 22281052@bjtu.edu.cn

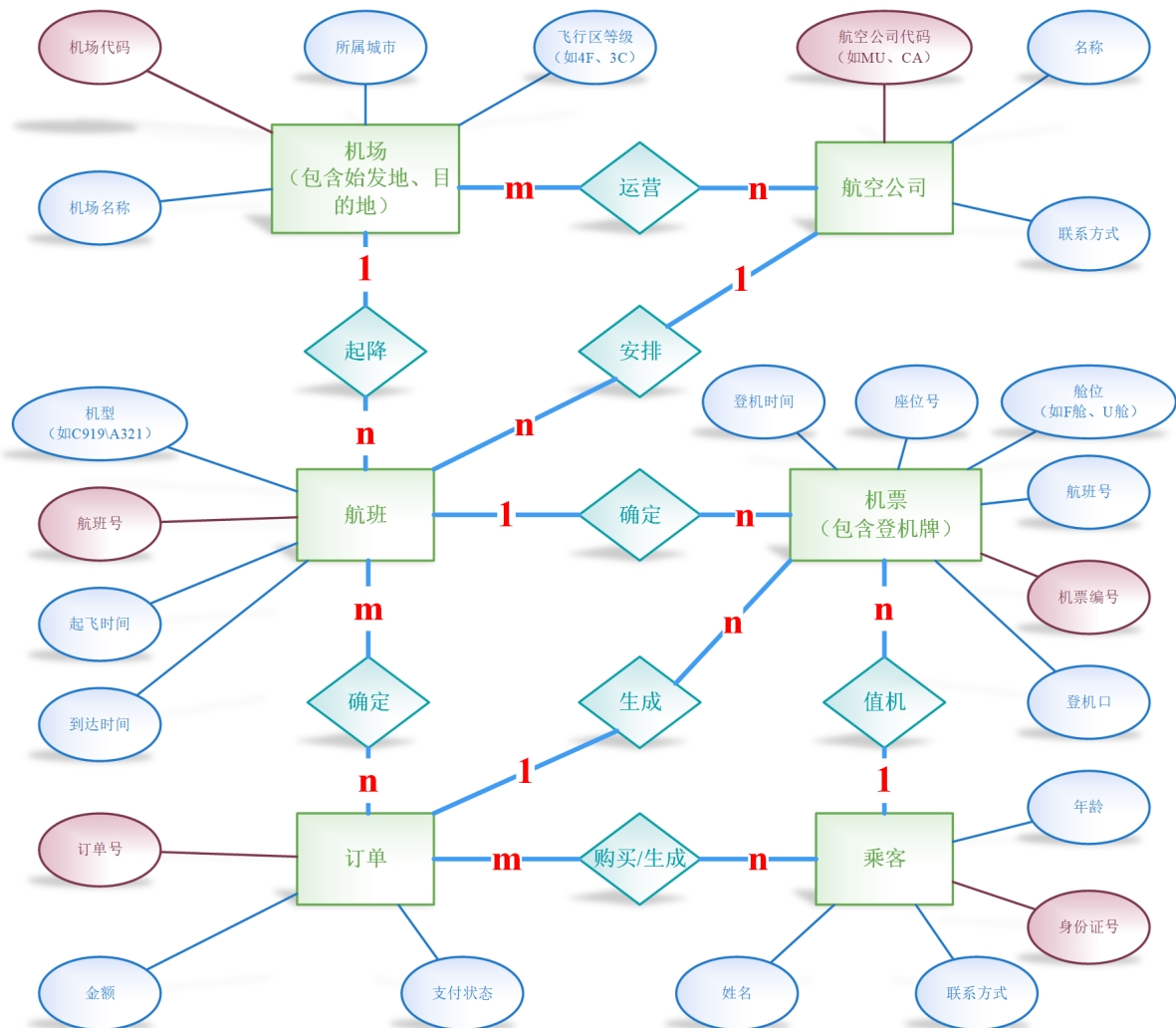


图 1: 航空公司航班查询业务模型 E-R 图, 来自于作业 6

1 题目 1：备份与日志初步实验

题目要求：

- 了解你所使用的数据库平台的单表数据备份和整库备份方法，进行相应备份操作，并尝试利用备份数据在另一个机器上恢复数据，并在实验报告中描述上述过程。
- 掌握数据库日志的概念，并说明数据备份、日志与故障恢复之间的关系。
- 查阅资料，在你所使用的数据库中找到能记录数据修改操作的日志文件，针对某个表执行插入或修改操作，请在相应日志文件中找对应的插入或修改操作日志记录，至少解释其中的一条日志数据样例。

1.1 数据备份与恢复方法

数据备份是任何数据库管理策略中的首要环节，它是防范数据丢失的最后一道防线，也为数据审计、系统迁移等提供了基础。MySQL 提供了多种备份方法，主要可分为逻辑备份和物理备份。本实验主要采用逻辑备份方式，利用 MySQL 自带的 `mysqldump` 工具进行操作，因其具备良好的跨平台兼容性和数据可读性。

1.1.1 备份原理与工具概述

`mysqldump` 是 MySQL 官方提供用于执行逻辑备份的工具。其核心原理是连接至目标 MySQL 服务器，读取指定的数据库对象（如表、视图、存储过程等）的结构定义和其中包含的数据。随后，它将这些信息转化为一组标准的 SQL 语句（如 `CREATE TABLE` 用于重建表结构，`INSERT INTO` 用于填充数据），并将这些 SQL 语句输出至指定文件或标准输出流。

• 逻辑备份优势：

1. **跨平台与版本兼容性：**输出为纯文本 SQL 文件，可在不同操作系统、不同硬件架构及不同 MySQL 版本（只要目标版本不低于备份版本）之间进行恢复，极大地提升了灵活性。
2. **数据可读性与可编辑性：**备份文件内容易于人类阅读和理解，必要时可手动修改特定数据或结构。
3. **细粒度恢复：**可以方便地选择性备份或恢复单个表、特定数据库对象，无需处理整个数据目录。

• 逻辑备份局限性：

- **恢复效率：**对于拥有海量数据的大型数据库而言，备份和恢复过程相对耗时。这主要是因为数据需要先从二进制格式转换为 SQL 文本，然后在恢复时又需经过 SQL 解析、执行以及数据写入等步骤，这些 I/O 和 CPU 开销较大。

除了 `mysqldump`, MySQL 还支持物理备份, 如 MySQL Enterprise Backup (商业版) 或 Percona XtraBackup (开源), 它们直接复制数据库数据文件, 速度更快, 常用于大型数据库的热备份。然而, 其恢复通常受限于相同的操作系统和 MySQL 版本, 并且输出文件是二进制的, 不便于人工审查。

为了确保在数据库运行期间进行备份时数据的一致性, 特别是针对 InnoDB 存储引擎, 我使用了 `--single-transaction` 选项。该选项在备份开始时利用 InnoDB 的 MVCC (多版本并发控制) 机制, 创建一个事务隔离的快照, 保证在备份过程中即使有数据变动, 备份文件中的数据也是当时的一致状态。此外, `--master-data=2` 选项至关重要, 它会在备份文件中记录此次备份完成时的二进制日志文件名及其位置 (Position), 这为后续进行精确的点时间恢复 (Point-in-Time Recovery, PITR) 提供了关键信息。

1.1.2 备份操作过程描述

本实验在 MySQL 9.0.1 版本环境下, 通过 Windows PowerShell 终端执行 `mysqldump` 命令进行备份操作。

1. **全数据库备份 (airlinedb):** 我执行了以下命令, 将整个 `airlinedb` 数据库的结构和所有数据备份到名为 `airlinedb_consistent_backup.sql` 的文件中。

```
1 mysqldump -u root -p airlinedb --single-transaction --master-  
   data=2 > airlinedb_consistent_backup.sql
```

在执行过程中, 系统会提示输入 `root` 用户的密码。命令执行成功后, 将会在当前目录下生成一个包含数据库所有对象和数据的 SQL 文本文件。

2. **单表备份 (Passenger 表):** 为了演示 `mysqldump` 的灵活性, 我还执行了仅备份 `Passenger` 表的命令。

```
1 mysqldump -u root -p airlinedb Passenger > passenger_backup.sql
```

这会生成一个只包含 `Passenger` 表结构和数据的 `passenger_backup.sql` 文件。

1.1.3 恢复操作过程描述

数据恢复是将备份数据重新载入数据库以重建特定时间点状态的过程。本次实验在另一台电脑上, MySQL 9.3.0 版本环境下, 通过 Windows PowerShell 终端进行数据恢复, 以验证备份的完整性和有效性。

1. **备份文件传输:** 首先, 通过文件拷贝的方式将生成的 `airlinedb_consistent_backup.sql` 备份文件传输到目标恢复机器的合适位置。
2. **创建目标数据库:** 在目标机器上, 登录 MySQL 客户端, 并创建一个用于恢复数据的新数据库。此举确保恢复过程在一个干净且独立的环境中进行, 避免对现有数据造成污染。

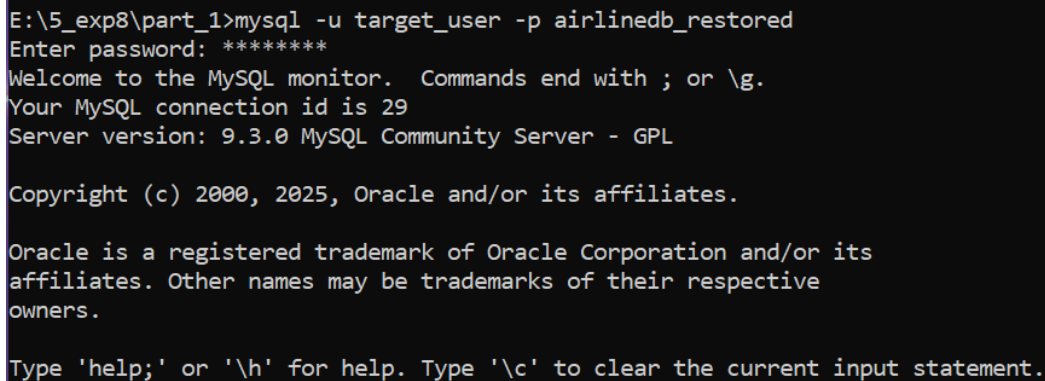
```
1 CREATE DATABASE airlinedb_restored;
```

由于需要展现独立用户的情形，在 MySQL 客户端中需要创建用户并授予相关权限。sql 语句如下：

```
1 # xxx为密码
2 CREATE USER 'target_user'@'localhost' IDENTIFIED BY 'xxx';
3 FLUSH PRIVILEGES;
4 # 为用户授予权限
5 GRANT ALL PRIVILEGES
6     ON airlinedb_restored.*
7     TO 'target_user'@'localhost';
8 GRANT SYSTEM_USER ON *.* TO 'target_user'@'localhost';
9 FLUSH PRIVILEGES;
```

3. **执行恢复命令：**退出 MySQL 客户端，在操作系统命令行中执行以下命令。该命令利用 MySQL 客户端工具的重定向功能，将备份文件中的所有 SQL 语句作为输入，执行到 airlinedb_restored 数据库中。

```
1 mysql -u target_user -p airlinedb_restored <
   airlinedb_consistent_backup.sql
```



```
E:\5_exp8\part_1>mysql -u target_user -p airlinedb_restored
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 29
Server version: 9.3.0 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```

图 2: MySQL 恢复命令执行示例

其中，target_user 是目标 MySQL 服务器上具有足够权限的用户。

4. **恢复验证：**恢复完成后，重新登录 MySQL 客户端，切换到 airlinedb_restored 数据库。通过执行简单的查询（如 SELECT COUNT(*) 从重要表中统计记录数，或 SELECT * 语句随机查看部分数据），来验证数据库结构是否完整，数据是否已成功导入。

```
1 # 查询恢复的表
2 SHOW TABLES;
3 # 查询测试
4 USE airlinedb_restored;
```

```

5 SELECT COUNT(*) FROM Passenger;
6 SELECT * FROM Flight LIMIT 5;

```

```

C:\Windows\System32>mysql -u target_user -p airlinedb_restored
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 30
Server version: 9.3.0 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SHOW TABLES;
+-----+
| Tables_in_airlinedb_restored |
+-----+
| airline                      |
| airlineflightcount           |
| airport                     |
| flight                      |
| flightsum                   |
| flighttime                  |
| orderinfo                   |
| passenger                   |
| passfuturefli               |
| passhistoryfli              |
| ticket                      |
| triggerlog                  |
| userinfo                    |
+-----+
13 rows in set (0.011 sec)

```

图 3: 恢复后数据验证结果 1

```

mysql> USE airlinedb_restored;
Database changed
mysql> SELECT COUNT(*) FROM Passenger;
+-----+
| COUNT(*) |
+-----+
|      53 |
+-----+
1 row in set (0.042 sec)

mysql> SELECT * FROM Flight LIMIT 5;
+-----+-----+-----+-----+-----+-----+-----+
| flight_number | air_model | departure_time | arrival_time | departure_airport | arrival_airport | airline_code |
+-----+-----+-----+-----+-----+-----+-----+
| CA1112        | Boeing737 | 2024-04-25 20:45:09 | 2024-04-25 23:45:09 | PVG               | PEK              | CA           |
| CA1234        | Boeing737 | 2024-04-10 08:00:00 | 2024-04-10 10:30:00 | PEK               | PVG              | CA           |
| CA2809        | Boeing737 | 2024-04-17 09:12:05 | 2024-04-17 13:12:05 | PVG               | PEK              | CA           |
| CA7836        | Boeing737 | 2024-04-29 18:14:09 | 2024-04-29 22:14:09 | PVG               | PEK              | CA           |
| MU4241        | AirbusA320 | 2024-04-29 17:01:06 | 2024-04-29 19:01:06 | PVG               | PEK              | MU           |
+-----+-----+-----+-----+-----+-----+-----+
5 rows in set (0.008 sec)

```

图 4: 恢复后数据验证结果 2

实验结果表明，通过 mysqldump 生成的逻辑备份文件能够有效地用于数据库的恢复，无论是结构还是数据都能被准确地重建。这验证了逻辑备份在数据库灾难恢复策略中的实用性。

1.2 数据库日志概念及其与故障恢复的关系

数据库日志作为数据库管理系统的核心组成部分，是一系列不可或缺的记录文件。它们详尽地记录了数据库系统的内部运行状态、事务的完整生命周期以及对数据的每一次变更。这些日志文件是数据库系统实现高可靠性、数据持久性与可恢复性的根本保障，它们的重要性贯穿于数据管理的各个环节。

1.2.1 数据库日志类型及其功能

在 MySQL 数据库系统中，存在多种不同类型的日志文件，每种日志都承载着特定的管理和故障恢复功能：

- **错误日志 (Error Log)**：记录 MySQL 服务器启动、运行、关闭过程中的各类事件，包括严重的错误、警告和一般性通知。它是诊断服务器健康状况和排查系统级故障的首要依据。
- **通用查询日志 (General Query Log)**：如果启用，此日志将详尽地记录所有客户端连接、执行的每条 SQL 语句（包括查询和数据修改），以及连接断开的信息。它对于调试应用程序和进行全面审计具有重要价值，但因巨大的 I/O 开销，不建议在生产环境中长期启用。
- **慢查询日志 (Slow Query Log)**：该日志专门记录执行时间超过预设阈值(`long_query_time` 参数)的所有 SQL 查询。它是数据库性能优化的重要工具，通过分析慢查询日志，可以识别出效率低下的 SQL 语句，进而对其进行优化或创建合适的索引。
- **二进制日志 (Binary Log, Binlog)**：这是 MySQL 实现数据复制 (Replication) 和点时间恢复 (Point-in-Time Recovery, PITR) 的核心日志。Binlog 以“事件” (events) 的形式，逻辑性地记录所有对数据库数据或结构产生修改的操作，例如插入、更新、删除数据，以及创建或修改表结构等。它记录的是**逻辑操作**，即“发生了什么修改”和“哪些数据被影响”。Binlog 具有前滚 (Roll Forward) 功能，能够将数据库从一个历史备份点推进到任意指定的时间点，甚至到故障发生前的最新状态。
- **重做日志 (Redo Log, 特指 InnoDB 存储引擎)**：与 Binlog 记录逻辑操作不同，Redo Log 记录的是 InnoDB 存储引擎层对数据页面所做的**物理更改**。其根本目的在于保证事务的**持久性 (Durability)**，并实现**崩溃恢复 (Crash Recovery)**。当数据库服务器在事务提交后、但相关数据尚未完全刷新到磁盘之前意外崩溃时，Redo Log 能够确保这些已提交的更改不会丢失。在数据库系统重启时，InnoDB 会扫描并重新应用 Redo Log 中记录的更改，以确保数据文件恢复到崩溃前的一致状态。

1.2.2 数据备份、日志与故障恢复的协同作用

数据备份、二进制日志和重做日志并非孤立存在，它们之间形成了一个紧密协作的体系，共同构建了数据库多层次的故障恢复策略。这种协同作用使得数据库在面对从轻微故障到严重灾难的各种情境时，都能够被可靠地恢复到一致且尽可能最新的状态。

数据备份作为所有恢复操作的基点，提供了一个数据库在**某个特定时间点的完整静态快照**。当遭遇硬件故障、数据文件损坏、人为误操作（如意外删除大量数据）或整个系统崩溃等灾难性事件时，最近的有效备份是找回数据、重新构建数据库系统的首要手段。它为后续的恢复过程提供了一个已知且一致的“基线”版本。

二进制日志 (Binlog)在备份的基础上发挥着至关重要的作用。它详尽记录了从基础备份完成之后数据库中发生的所有数据变更。这使得数据库能够从这个基础备份点“前

滚”（Roll Forward）到备份之后的任意精确时间点（例如，误操作发生前一秒），甚至是故障发生前的最新状态。例如，如果一个关键数据表在凌晨 3 点的全量备份之后、上午 10 点被错误地删除，通过恢复到凌晨 3 点的备份，然后顺序应用从 3 点到 9 点 59 分 59 秒的 Binlog，就可以精确地避免了误删除操作。Binlog 不仅是实现灵活的点时间恢复（PITR）的核心，也是数据库复制（Replication）技术的基石。

重做日志 (Redo Log) 主要用于保障 InnoDB 存储引擎在服务器意外崩溃后的数据一致性和持久性。当数据库服务器在未正常关闭（如突然断电或进程被强制终止）的情况下停止时，内存中已提交但尚未完全写入磁盘的事务更改可能会面临丢失的风险。Redo Log 通过“写前日志”（Write-Ahead Logging, WAL）机制，确保在数据更改写入磁盘数据文件之前，其对应的日志记录已先行且持久地写入 Redo Log 文件。在数据库系统重启时，InnoDB 存储引擎会读取 Redo Log，并根据其中的记录重新应用（重做）所有已提交但未写入磁盘的事务更改，确保数据文件恢复到崩溃前的一致状态。Redo Log 是保障单个 InnoDB 实例在非正常关闭后数据不丢失的核心机制，是事务原子性和持久性的底层保障。

综上所述，**数据备份**提供了恢复的宏观基线，**二进制日志**提供了从基线到目标恢复点（或最新点）的精确数据变更历史，实现了灵活的“前滚”；而**重做日志**则保障了数据库在系统崩溃后的内部一致性与持久性。这三者紧密协作，共同形成了多层次、高效率的恢复方案，能够全面应对从轻微的数据不一致到严重的系统级灾难等各种数据库故障场景，确保数据库系统的高可用性与数据完整性。

1.3 Binlog 记录数据修改操作的实验与分析

1.3.1 实验环境配置与 Binlog 状态验证

根据 my.ini 配置文件，MySQL 服务器已明确启用二进制日志功能。其关键配置参数包括：log-bin="ROBIN-bin"（指定 Binlog 文件前缀）、server-id=1（服务器 ID）、binlog_format=ROW（指定为行格式）以及 datadir=Binlog 文件存放路径。

通过 MySQL 客户端执行 SHOW VARIABLES LIKE 'log_bin'; 等命令，确认了 Binlog 的开启状态为 ON，且格式为 ROW，与配置文件一致。

```
mysql> SHOW VARIABLES LIKE 'log_bin';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| log_bin       | ON    |
+-----+-----+
1 row in set, 1 warning (0.12 sec)

mysql> SHOW VARIABLES LIKE 'binlog_format';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| binlog_format | ROW   |
+-----+-----+
1 row in set, 1 warning (0.00 sec)

mysql> SHOW VARIABLES LIKE 'datadir';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| datadir       | S:\MySQL\Data\MySQL Server 9.0\Data\ |
+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

图 5: Binlog 配置变量检查结果

使用 SHOW BINARY LOG STATUS; 命令可以查看当前正在写入的 Binlog 文件及其当前写入位置。

```
mysql> SHOW BINARY LOG STATUS;
+-----+-----+-----+-----+-----+
| File           | Position | Binlog_Do_DB | Binlog_Ignore_DB | Executed_Gtid_Set |
+-----+-----+-----+-----+-----+
| ROBIN-bin.000023 |      158 |              |                  |                  |
+-----+-----+-----+-----+-----+
1 row in set (0.02 sec)
```

图 6: SHOW BINARY LOG STATUS 命令结果

1.3.2 执行数据修改操作并使用 mysqlbinlog 解析

在确认 Binlog 正常运行后，我在 airlinedb.Passenger 表上执行了典型的 DML (Data Manipulation Language) 操作：插入一条新记录和更新一条现有记录。

1. **执行插入操作：**我向 Passenger 表中插入一条新的乘客记录。

```
1 INSERT INTO Passenger (id_card, passname, age, phone)
2 VALUES ('510101200001019000', '测试用户 Binlog', 35, '13900001234
   ');
```

2. **执行更新操作：**我随后更新了 Passenger 表中一条现有记录的电话和年龄。

```
1 UPDATE Passenger
2 SET phone = '13800000001'
3 WHERE id_card = '110101199001011234';
```

在每次操作完成后，使用下方语句，记录了 binlog 记录的位置：返回的 Binlog 文件名和其对应的结束位置 (Position)，这些位置信息是后续使用 mysqlbinlog 工具精确解析的关键。

```
1 SHOW BINARY LOG STATUS;
```

```
mysql> USE airlinedb;
Database changed
mysql> INSERT INTO Passenger (id_card, passname, age, phone)
-> VALUES ('510101200001019000', '测试用户 Binlog', 35, '13900001234');
Query OK, 1 row affected (0.09 sec)

mysql> UPDATE Passenger
-> SET phone = '13800000001'
-> WHERE id_card = '110101199001011234';
Query OK, 1 row affected (0.13 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SHOW BINARY LOG STATUS;
+-----+-----+-----+-----+-----+
| File           | Position | Binlog_Do_DB | Binlog_Ignore_DB | Executed_Gtid_Set |
+-----+-----+-----+-----+-----+
| ROBIN-bin.000023 |      1459 |              |                  |                  |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

图 7: binlog 测试插入与更新

接着，在操作系统命令行中使用 `mysqlbinlog` 工具对 Binlog 文件进行解析。由于 Binlog 格式为 ROW，它记录的是数据行的具体变动，而非原始 SQL 语句。

```
# at 472
#250604 15:34:54 server id 1  end_log_pos 563 CRC32 0x6df84d22  Write_rows: table id 85
# at 563
#250604 15:34:54 server id 1  end_log_pos 727 CRC32 0x8cd8dc09  Write_rows: table id 88 flags: STMT_END_F

BINLOG '
Hvc/aBMBAAAAASgAAAIkBAAAAAFUAAAAAAMACWFpcmxpbmVkJGAKJcGFzc2VuZ2VyaAQPdWMPBIAA
yACQAQgBAQACA/z/AORsE6s=
Hvc/aBMBAAATwAAANGbAAAAAFgAAAAAAMACWFpcmxpbmVkJGAKdHJpZ2d1cmxvZWAGAw8PD/wS
CMgAKADIAAIAOAEBAADID/P8akSFxpg==
Hvc/aB4BAAAAwAAADMCACAAAAAFUAAAAAAMACWFpcmxpbmVkJGAKdHJpZ2d1cmxvZWAGAw8PD/wS
leeUqOaIt0JpbmVxZyMAAAALADeZOTAwMDAxMjM0Ik34bQ==
Hvc/aB4BAAAApAAAAACCAAAAFgAAAAAAMACWFpcmxpbmVkJGAKdHJpZ2d1cmxvZWAGAw8PD/wS
MTAxMjAwMDAxMDE5MDAwUCBQZGZzc2VuZ2VyaAQPdWMPBIAA
MDE5MDAwLCBQYWI1OiDmtYvor5XnlKjmiLDcAW5sb2csIEFnZTogMzWztsj4tgnc2lw=
/*!*/
```

图 8: mysqlbinlog 结果 (INSERT 操作示例)

```
# at 1081
#250604 15:35:01 server id 1 end_log_pos 1209 CRC32 0x0ac65297 Update_rows: table id 85
# at 1209
#250604 15:35:01 server id 1 end_log_pos 1428 CRC32 0x2449ac46 Write_rows: table id 88 flags: STMT_END_F

BINLOG '
Jfc/aBMBAAAAAGSaaA0oDAAAAAFUAAAAAAAAACWfPcmxpbmVkJGAgJcGfzc2VuZ2VyAAQPDwMPBIAA
yAQQAQgBAQACA/z/AF/DW60=
Jfc/aBMBAAAATtAAADkEAAAAAFGAAAAAACWfPcmxpbmVkJGAKdHjPz2dlcmxvZWAGA8PD/wS
CMgAKADIAATAOEBAAD/P8AHP+r3fq==
Jfc/aBS8AAAAAGAAAlkEAAAAAFUAAAAAAAAAGAE//SAEjExMDEwMTE5OTAwMTAxMTIzNahaGFu
Z3Nhbh4AAAAALADEzODAwMTFM4MDAwABlIXtAXMDEwXOtKwMDEwMTEyYmZlYW4eAAAAcWAx
MzgMdAwMDAwMZDsxgo=
Jfc/aB4BAAAA2AAAJQFAAAAAAFGAAAAAAEAAG/agWANAAACVBhc3B1bnRldlgZVUERBEUSMTEw
MTAxMTIxMDEwMDEwMjoiABqYXNZW5nZXIgdXBkYXRlZC4SUQ6IDExMDEwMTE5OTAwMTAxMTIz
NC4gaTmfZSBmc9tICJaagFuZ3NhbiIGdXBkYXRlZCB0byAiMWhhbmdmdzYw4ILiBQagUzSBmc9t
IClIZmzgMdEzODAwMCIgdXBkYXRlZCB0byAiMTM4MDAwMDAwMDEimbBI+MFGRkkk
/*!*/;
```

图 9: mysqlbinlog 结果 (UPDATE 操作示例)

为了将这些二进制数据解码为可读格式，我使用了 `--verbose` 选项。

```
# at 772
#250604 15:34:54 server id 1 end_log_pos 563 CRC32 0x6df84d22 Write_rows: table id 85
# at 563
#250604 15:34:54 server id 1 end_log_pos 727 CRC32 0x8dc8dc09 Write_rows: table id 88 flags: STMT_END_F

BINLOG '
Hvc/aBMBAAAAASgAAATkBAAAAAFUAAAAAAMACWfpcmxpbmVkYgAJcGFzc2VuZ2VYAAQPDwMPBIAA
yACQAQgBAQACA/z/AORsE6s=
Hvc/aBMBAAAAATwAAANgBAAAAAFgAAAAAAMACWfpcmxpbmVkYgAKdHJpZ2dlcmxvZWAgaW8PD/wS
CMgAKADIAATAOAEBAID/P8akSFxpg==
Hvc/aB4BAAAAWwAAADMCAAAAAFUAAAAAAGaE/wASNTEwMTAxMjAwMDAwMDE5MDAwEual+i+v
leeUq0aIt0JpbmvZyMAAAALADEz0TAwMDAxMjM0Ikd3bQ==
Hvc/aB4BAAAApAAAAACCAAAAFgAAAAAAEAAGAC/wAMAAAAACVBhc3NlbmdldGZJTlNFU1QSNTEw
MTAxMjAwMDAwMDE5MDAwUQB0ZXcgcGFzc2VuZ2VYIGluc2VydGVkLiBjRDQNTewMTAxMjAwMDAw
MDAwMDAwLDBOYWV1I0lMdYvor5XnlKjmiLdCaw5sb2csIEFnZTogMzZlZ3tj4tgnC2lw=
'/*!*/;

### INSERT INTO `airlinedb`.`passenger`
### SET
### @1='510101200001019000'
### @2=' 嫫嫫痲縻儿垌Binlog'
### @3=35
### @4='13900001234'
### INSERT INTO `airlinedb`.`triggerlog`
### SET
### @1=12
### @2='Passenger'
### @3='INSERT'
### @4='510101200001019000'
### @5='New passenger inserted. ID: 510101200001019000, Name: 嫫嫫痲縻儿垌Binlog, Age: 35'
### @6='2025-06-04 15:34:54'
# at 727
#250604 15:34:54 server id 1 end_log_pos 758 CRC32 0x3385d74c Xid = 11
COMMIT/*!*/;
```

图 10: mysqlbinlog 解析结果 (INSERT 操作示例)

```

# at 1081
#250604 15:35:01 server id 1 end_log_pos 1209 CRC32 0x0ac65297 Update_rows: table id 85
# at 1209
#250604 15:35:01 server id 1 end_log_pos 1428 CRC32 0x2449ac46 Write_rows: table id 88 flags: STMT_END_F

BINLOG
Jfc/aBMBAAAAsgAAAsDAAAAAFUAAAAAAMACWFpcmxpbmVhYyAkcGFzc2VhZ2VyaAQPDwMPB1AA
jAQQAQBAQAQzAF/DH6g=
Jfc/aBMBAAAAsgAAAsDAAAAAFUAAAAAAMACWFpcmxpbmVhYyAkcGFzc2VhZ2VhZ2VyaAQPDwMPB1AA
CMgAKADIAIAIAOAEBAID/PSAHP3fQ=
Jfc/aBMBAAAAsgAAAsDAAAAAFUAAAAAAMACWFpcmxpbmVhYyAkcGFzc2VhZ2VhZ2VyaAQPDwMPB1AA
Z3Nhbhh4AAALADEzODAwMTM4MDAwABIxMTAxMDEwOTkwMDEwMTEyMzQlWmhbmddzYW4eAAACwAx
MzgWMDAwMDAwMzdsXgo=
Jfc/aBMBAAAAsgAAAsDAAAAAFUAAAAAAMACWFpcmxpbmVhYyAkcGFzc2VhZ2VhZ2VyaAQPDwMPB1AA
MTAxMTk5MDAwMDEwMTM4MDAwABIxMTAxMDEwOTkwMDEwMTEyMzQlWmhbmddzYW4eAAACwAx
NC4gTmFtZSBmc9tICJaaGFuZ3Nhb1IgdXBkYXR1ZCB0byA1WmhbmddzYW4eAAACwAx
ICJkMzgWMDAwMDEwMTM4MDAwABIxMTAxMDEwOTkwMDEwMTEyMzQlWmhbmddzYW4eAAACwAx
/*!*/;
### UPDATE `airlinedb`.`passenger`
### WHERE
### @1='110101199001011234'
### @2='Zhangsan'
### @3=30
### @4='13800138000'
### SET
### @1='110101199001011234'
### @2='Zhangsan'
### @3=30
### @4='13800000001'
### INSERT INTO `airlinedb`.`triggerlog`
### SET
### @1=13
### @2='Passenger'
### @3=UPDATE
### @4='110101199001011234'
### @5='Passenger updated. ID: 110101199001011234. Name from "Zhangsan" updated to "Zhangsan". Phone from "13800138000" updated to "13800000001"'
### @6='2025-06-04 15:35:01'
# at 1428
#250604 15:35:01 server id 1 end_log_pos 1459 CRC32 0xde0847b9 Xid = 13
COMMIT/*!*/;
SET @@SESSION.GTID_NEXT= 'AUTOMATIC' /* added by mysqlbinlog */ /*!*/;
DELIMITER ;
# End of log file
/*!50003 SET COMPLETION_TYPE=@OLD_COMPLETION_TYPE*/;
/*!50530 SET @@SESSION.PSEUDO_SLAVE_MODE=0*/;

```

图 11: mysqlbinlog 解析结果 (UPDATE 操作示例)

1.3.3 日志数据样例解释

以插入操作的 Binlog 解析输出片段为例，其核心部分及解释如下：

```

1 # at 472                -- 此事件在 binlog 文件中的起始位置
2 #250604 15:34:54 server id 1 end_log_pos 563 CRC32 0x6df84d22
   Write_rows: table id 85
3 # database airlinedb    -- 变动发生的数据库
4 # table `airlinedb`.`passenger` -- 变动发生的表
5 # V1
6 # insert                -- 操作类型：插入
7 # ### INSERT INTO `airlinedb`.`passenger`
8 # ### SET
9 # ### @1='510101200001019000' -- 列 1 (id_card) 的值
10 # ### @2='测试用户 Binlog' -- 列 2 (passname) 的值
11 # ### @3=35              -- 列 3 (age) 的值
12 # ### @4='13900001234' -- 列 4 (phone) 的值

```

- **# at 472**: 该事件在 Binlog 文件中的起始偏移量为 472 字节。
- **#250604 15:34:54 server id 1**: 事件发生的时间戳（精确到秒），表示 2025 年 6 月 4 日 15 时 34 分 54 秒，以及执行此操作的 MySQL 服务器的唯一标识 ID 为 1。
- **end_log_pos 563**: 该事件在 Binlog 文件中的结束位置为 563 字节。下一个事件将从该位置开始。
- **Write_rows: table id 85**: 表明这是一个写入行事件（对应 DML 中的 INSERT 操作），作用于数据库内部标识为 85 的表。

- `# database airlinedb` 和 `# table `airlinedb`.`passenger``: 明确指出此操作作用于 `airlinedb` 数据库的 `passenger` 表。
- `insert`: 再次确认了此事件的 DML 操作类型为插入。
- `### SET @1='510101200001019000' ...`: 这是 `mysqlbinlog --verbose` 对 ROW 格式数据进行解码后的详细显示。它以“列名”和“值”的形式列出了被插入的行的具体数据。例如, `@1` 代表 `passenger` 表的第一列 (通常是 `id_card`), 其值为 `'510101200001019000'`, 以此类推, 完整记录了新插入行的所有列数据。

对于更新操作 (`Update_rows` 事件), Binlog 记录会包含 `BEFORE:` 和 `AFTER:` 两部分数据, 分别表示更新前和更新后的行数据, 从而能够精确地追踪数据的具体变化。这种行级的记录方式确保了数据恢复或复制的精确性和可靠性。

2 题目 2：并发控制实验

题目要求：

- 通过取消所用 *DBMS* 的查询分析器的自动提交功能，创建两个不同用户，分别登录查询分析器，同时打开两个客户端；
- 通过 *SQL* 语言设计具体例子展示不同隔离级别的应用场景，验证各种隔离级别的并发控制效果，即是否存在并发操作带来的数据不一致问题，包括丢失修改、不可重复读和读“脏”数据等。

在多用户并发访问数据库的场景中，如何确保数据的正确性和一致性是数据库管理系统面临的核心挑战。并发控制旨在通过各种机制（如锁、多版本并发控制等）来协调并发事务的执行，从而避免数据不一致问题。事务隔离级别是 ANSI/ISO SQL 标准定义的一组概念，用于描述事务并发执行时彼此隔离的程度。MySQL 作为主流的关系型数据库，提供了四种标准的事务隔离级别，每种级别通过不同的策略来平衡事务隔离性与系统并发性能。本实验旨在通过模拟两个并发客户端，在不同隔离级别下观察并验证这些隔离级别对并发问题（包括脏读、不可重复读、幻读和丢失修改）的控制效果。

2.1 事务隔离级别概述

事务隔离级别是数据库事务管理中的重要概念，它定义了一个事务在并发环境中对其他事务执行的更改的可见性，以及本事务对其他事务的更改的可见性。SQL 标准定义了四种隔离级别，隔离程度从低到高，它们各自解决不同类型的并发问题。

2.1.1 READ UNCOMMITTED (读未提交)

这是最低的隔离级别。在此级别下，一个事务可以读取另一个事务尚未提交的数据（也称为“脏数据”）。由于读取的数据可能最终被回滚，因此这种数据是不可靠的。它不提供任何锁机制来阻止读取未提交的数据，因此并发性能最高，但数据一致性最差。

- **允许的问题：**脏读 (Dirty Read)、不可重复读 (Non-repeatable Read)、幻读 (Phantom Read)、丢失修改 (Lost Update)。

2.1.2 READ COMMITTED (读已提交)

这是许多数据库系统（如 Oracle、PostgreSQL）的默认隔离级别。在此级别下，一个事务只能读取其他事务已经提交的数据，这有效避免了“脏读”问题。然而，在同一个事务中，如果两次读取同一行数据，而期间另一事务修改并提交了该行，那么两次读取的结果可能会不同，导致“不可重复读”。

- **允许的问题：**不可重复读 (Non-repeatable Read)、幻读 (Phantom Read)、丢失修改 (Lost Update)。
- **解决的问题：**脏读。

2.1.3 REPEATABLE READ (可重复读)

这是 MySQL InnoDB 存储引擎的默认隔离级别。在此级别下，一个事务在开始时创建一个数据快照 (Snapshot)，事务在执行期间（包括多次读取同一行）总是能看到这个快照版本的数据，从而避免了“不可重复读”问题。根据 SQL 标准，REPEATABLE READ 隔离级别仍然允许“幻读”的发生。然而，MySQL InnoDB 存储引擎对其进行了加强实现：通过使用“Next-Key Locks”（行锁与间隙锁的组合），它能够有效地阻止其他事务在查询范围内插入新行，从而在大多数情况下防止了“幻读”。

- **允许的问题：**(SQL 标准允许幻读，但 MySQL InnoDB 通常能解决) 丢失修改 (Lost Update)。
- **解决的问题：**脏读、不可重复读、幻读（在 MySQL InnoDB 中）。

2.1.4 SERIALIZABLE (串行化)

这是最高的隔离级别。在此级别下，事务的并发执行结果与这些事务按某种串行顺序执行的结果相同。为了达到这一目标，数据库系统通常会通过对所有读操作和写操作加锁来实现。所有的 SELECT 语句都转换为 SELECT ... FOR SHARE 或 SELECT ... FOR UPDATE 等加锁读，所有写入操作都会获得排他锁。这彻底杜绝了所有并发问题，包括脏读、不可重复读和幻读。然而，高隔离性也意味着并发性能的显著下降，因为它可能导致大量的锁等待和死锁。

- **允许的问题：**无。
- **解决的问题：**脏读、不可重复读、幻读、丢失修改。

2.1.5 事务隔离级别与并发问题对照表

下表总结了四种事务隔离级别对于三大经典并发问题的处理能力（Y 表示可能发生，N 表示已解决）。值得注意的是，此表基于 SQL 标准，并特别指出 MySQL InnoDB 存储引擎在 REPEATABLE READ 隔离级别下对幻读的加强处理。

表 1: 事务隔离级别与并发问题对照

隔离级别	脏读 (Dirty Read)	不可重复读 (Non-repeatable Read)	丢失修改 (Lost Update)	幻读 (Phantom Read)
READ UNCOMMITTED	Y	Y	Y	Y
READ COMMITTED	N	Y	Y	Y
REPEATABLE READ	N	N	N (MySQL InnoDB) / Y (SQL 标准)	Y
SERIALIZABLE	N	N	N	N

2.2 实验环境与准备

实验环境搭建在 MySQL 9.0.1 数据库之上，并发客户端的模拟通过 Python 语言及其 pymysql 库以及 threading 模块实现。我创建了一个简单的 TestBalance 表来模拟账户余额，作为实验对象。

2.2.1 测试表 TestBalance

在实验开始前，数据库中需要创建 TestBalance 表，并预设少量初始数据。每次测试场景启动时，该表的数据都会被重置到统一的初始状态，以确保实验结果的独立性和可重复性。

```
1 mysqldump -u root -p airlinedb Passenger > passenger_backup.sql

1 CREATE TABLE TestBalance (
2     id INT PRIMARY KEY,
3     account_name VARCHAR(50),
4     balance DECIMAL(10, 2)
5 );
6 -- 初始数据 (id=1, 'Alice', 1000.00; id=2, 'Bob', 500.00) 由 Python 脚本在每次测试前插入。
```

2.2.2 并发模拟客户端

Python 脚本通过创建两个独立的线程来模拟两个并发的客户端 A 和客户端 B。每个线程都会建立自己的数据库连接，并独立地执行一系列数据库操作。为了精确控制并发时序，线程间通过 threading.Event 对象进行信号传递和等待，确保操作严格按照预设的步骤发生。数据库连接的 autocommit 特性被设置为 False，使得事务的提交或回滚需要显式执行。每个测试场景开始时，会话的隔离级别会被动态设置为当前场景所需的级别。

```
def create_db_connection(db_config):
    """创建并返回一个数据库连接，禁用自动提交"""
    try:
        conn = pymysql.connect(**db_config)
        conn.autocommit(False) # 禁用自动提交，手动控制事务
        return conn
    except pymysql.MySQLError as e:
        with print_lock:
            print(f"数据库连接失败: {e}", file=sys.stderr)
```

图 12: Python 并发实验设置示意图

2.3 事务隔离级别与并发问题验证

这一部分将逐一演示在 MySQL 不同隔离级别下，各种并发问题（脏读、不可重复读、幻读和丢失修改）的发生与阻止情况。

2.3.1 脏读 (Dirty Read)

脏读指一个事务读取了另一个事务未提交的数据，而这个数据随后被回滚了。它是数据库事务中最低级别的一致性问题。

READ UNCOMMITTED 隔离级别下的表现 在 READ UNCOMMITTED 隔离级别下，事务不加锁，允许读取其他事务的未提交更改。

```
=====
开始测试场景：脏读 (Dirty Read) 演示
隔离级别：READ UNCOMMITTED
=====
数据库和TestBalance表已初始化，并插入初始数据。
[客户端B] 开始事务...
[客户端A] 开始事务...
[客户端A] 尝试更新账户 1，金额：-200.00
[客户端A] 成功更新账户 1。
[客户端A] 更新Alice余额为800.00，未提交。|
[客户端B] 查询账户 1 余额：800.00
[客户端A] 回滚事务，Alice余额将恢复为1000.00。
[客户端B] 查询账户 1 余额：1000.00
[客户端B] 提交事务。

测试场景：脏读 (Dirty Read) 演示 结束。
=====
```

图 13: READ UNCOMMITTED 脏读演示输出

在本次实验中，客户端 A 将 Alice 账户余额更新为 800.00 但尚未提交。客户端 B 在第一次查询时，读取到了这个由客户端 A 产生的未提交的 800.00。随后，客户端 A 回滚事务，Alice 账户恢复到初始的 1000.00。客户端 B 在第二次查询时，则读取到了 1000.00。这清晰地表明了脏读的发生，即客户端 B 读取了最终并未持久化到数据库的中间数据，数据在事务中途发生了不一致的变化。

防止脏读：READ COMMITTED、REPEATABLE READ 和 SERIALIZABLE READ COMMITTED 及更高级别的隔离级别都能有效防止脏读，因为它们只允许事务读取已提交的数据。

```
=====
开始测试场景：防止脏读
隔离级别：READ COMMITTED
=====
数据库和TestBalance表已初始化，并插入初始数据。
[客户端B] 开始事务...
[客户端A] 开始事务...
[客户端A] 尝试更新账户 1，金额：-200.00
[客户端A] 成功更新账户 1。
[客户端A] 更新Alice余额为800.00，未提交。
[客户端B] 查询账户 1 余额：1000.00
[客户端A] 回滚事务，Alice余额将恢复为1000.00。
[客户端B] 查询账户 1 余额：1000.00
[客户端B] 提交事务。

测试场景：防止脏读 结束。
=====
```

图 14: READ COMMITTED 防止脏读输出

实验结果显示，当客户端 A 更新 Alice 余额为 800.00 但未提交时，客户端 B 在 READ COMMITTED（或 REPEATABLE READ，SERIALIZABLE）隔离级别下的第一次查询时，依然读取到 1000.00，而非客户端 A 的未提交更改。即使客户端 A 随后回滚，客户端 B 的两次查询结果都保持为 1000.00。这验证了这些隔离级别成功阻止了脏读，确保了事务只能看到已提交的、可靠的数据。

2.3.2 不可重复读 (Non-repeatable Read)

不可重复读指在同一个事务中，两次读取同一行数据，得到的结果不同。这发生在另一个已提交的事务在两次读取之间修改了该行数据。

```
=====
开始测试场景：不可重复读 (Non-repeatable Read) 演示
隔离级别：READ COMMITTED
=====
数据库和TestBalance表已初始化，并插入初始数据。
[客户端B] 开始事务...
[客户端A] 开始事务...
[客户端A] 查询账户 1 余额：1000.00
[客户端B] 尝试更新账户 1，金额：-100.00
[客户端B] 成功更新账户 1。
[客户端B] 提交更新Alice余额到900.00。
[客户端A] 查询账户 1 余额：900.00
[客户端A] 提交事务。

测试场景：不可重复读 (Non-repeatable Read) 演示 结束。
=====
```

图 15: READ COMMITTED 不可重复读演示输出

READ UNCOMMITTED 和 READ COMMITTED 隔离级别下的表现 在 READ UNCOMMITTED 和 READ COMMITTED 隔离级别下，客户端 A 第一次查询 Alice 余额为 1000.00。随后客户端 B 更新 Alice 余额为 900.00 并提交。客户端 A 在同一事务中第二次查询时读取到了 900.00。同一个事务内两次查询结果不同，表明不可重复读在这两个隔离级别下均会发生。这是因为这两个隔离级别允许事务读取其他事务已提交的最新版本数据。

防止不可重复读：REPEATABLE READ 和 SERIALIZABLE REPEATABLE READ 及更高级别的隔离级别通过维护事务开始时的数据快照来防止不可重复读。


```

=====
开始测试场景：防止不可重复读
隔离级别：REPEATABLE READ
=====
数据库和TestBalance表已初始化，并插入初始数据。
[客户端B] 开始事务...
[客户端A] 开始事务...
[客户端A] 查询账户 1 余额：1000.00
[客户端B] 尝试更新账户 1，金额：-100.00
[客户端B] 成功更新账户 1。
[客户端B] 提交更新Alice余额到900.00。
[客户端A] 查询账户 1 余额：1000.00
[客户端A] 提交事务。

测试场景：防止不可重复读 结束。
=====

```

图 16: REPEATABLE READ 防止不可重复读输出

实验结果显示，在 REPEATABLE READ 隔离级别下，客户端 A 的事务两次查询 Alice 余额都为 1000.00，即使客户端 B 已提交了更新。这验证了不可重复读被有效阻止，客户端 A 的事务在整个生命周期内都看到了数据的一致快照，未受其他事务提交的影响。

2.3.3 丢失修改 (Lost Update)

丢失修改指两个事务同时读取并基于旧值修改同一行数据，后提交的事务的修改覆盖了先提交的事务的修改，导致先提交的修改“丢失”。在实际数据库系统中，尤其是 MySQL 的 InnoDB 引擎，DML 操作（如 UPDATE）通常会加行级排他锁，从而自动阻止这种问题。

```

=====
开始测试场景：丢失修改 (Lost Update) 演示 (MySQL中会阻塞)
隔离级别：READ COMMITTED
=====
数据库和TestBalance表已初始化，并插入初始数据。
[客户端B] 开始事务...
[客户端A] 开始事务...
[客户端A] 查询账户 1 余额：1000.00
[客户端A] 计算新余额：1000.00 + 50.00 = 1050.00
[客户端B] 查询账户 1 余额：1000.00
[客户端B] 计算新余额：1000.00 + 100.00 = 1100.00
[客户端B] 尝试更新，期望会被客户端A阻塞。
[客户端B] 尝试更新账户 1，金额：+100.00
[客户端A] 尝试更新账户 1，金额：+50.00
[客户端A] 成功更新账户 1。
[客户端A] 提交事务。
[客户端B] 成功更新账户 1。
[客户端B] 提交事务。

测试场景：丢失修改 (Lost Update) 演示 (MySQL中会阻塞) 结束。
=====

```

图 17: READ COMMITTED/REPEATABLE READ 防止丢失修改输出

READ UNCOMMITTED 和 READ COMMITTED 隔离级别下对丢失修改的阻止 在实验中,即使在 READ UNCOMMITTED 或 READ COMMITTED 隔离级别下(此处以 READ COMMITTED 示例),客户端 A 和客户端 B 都读取了 Alice 的初始余额 1000.00,并尝试分别增加 50.00 和 100.00。理论上,如果两者都直接基于 1000.00 进行计算然后提交,可能发生丢失修改。然而,MySQL 的 UPDATE 语句会为更新的行加行级排他锁。实验结果显示,客户端 B 的更新操作被阻塞,直到客户端 A 提交事务并释放其行锁。待客户端 A 提交后,客户端 B 才得以执行并提交。最终 Alice 账户余额为 $1000 + 50 + 100 = 1150$ 。这表明 MySQL 的锁机制有效防止了丢失修改。

防止丢失修改:REPEATABLE READ 和 SERIALIZABLE REPEATABLE READ 和 SERIALIZABLE 隔离级别因其更严格的锁策略,自然能够防止丢失修改。在 REPEATABLE READ 级别下,实验结果与上述 READ COMMITTED 类似,两个客户端的更新操作被串行化执行,确保了每次更新都能基于前一次的最终结果,没有丢失任何修改。这再次印证了 MySQL 的 UPDATE 锁机制在这些隔离级别下的有效性。

2.3.4 幻读 (Phantom Read)

幻读指在同一个事务中,两次执行相同的范围查询,但第二次查询返回的行集发生了变化。这发生于另一个已提交的事务在第一次查询和第二次查询之间插入或删除了满足查询条件的新行。

```
=====
开始测试场景: 幻读 (Phantom Read) 演示
隔离级别: READ COMMITTED
=====
数据库和TestBalance表已初始化,并插入初始数据。
[客户端B] 开始事务...
[客户端A] 开始事务...
[客户端A] 查询余额大于0的账户数量...
[客户端A] 查询结果: 2 个账户余额大于0。
[客户端B] 尝试插入账户 3 (Charlie, 200.00)...
[客户端B] 成功插入账户 3。
[客户端B] 提交新账户插入。
[客户端A] 查询余额大于0的账户数量...
[客户端A] 查询结果: 3 个账户余额大于0。
[客户端A] *** 发生了幻读! 第二次查询看到了新插入的行。 ***
[客户端A] 提交事务。

测试场景: 幻读 (Phantom Read) 演示 结束。
=====
```

图 18: READ COMMITTED 幻读演示输出

READ UNCOMMITTED 和 READ COMMITTED 隔离级别下的表现 在 READ UNCOMMITTED 和 READ COMMITTED 隔离级别下,客户端 A 第一次查询余额大于 0 的账户数量为 2。随后客户端 B 插入新账户 (Charlie) 并提交。客户端 A 在同一事务中第二次查询时,账户

数量变为 3。这清晰地证实了幻读的发生，即客户端 A 看到了事务开始时不存在的“幻影”行，因为这两个隔离级别无法阻止其他事务在查询范围内插入新行。

防止幻读：REPEATABLE READ 和 SERIALIZABLE REPEATABLE READ 隔离级别在 SQL 标准中允许幻读，但在 MySQL InnoDB 中，其加强实现能够有效防止幻读。SERIALIZABLE 级别则通过更严格的锁策略完全阻止幻读。

```
=====
开始测试场景：防止幻读 (REPEATABLE READ)
隔离级别：REPEATABLE READ
=====

数据库和TestBalance表已初始化，并插入初始数据。
[客户端B] 开始事务...
[客户端A] 开始事务...
[客户端A] 查询余额大于0的账户数量...
[客户端A] 查询结果：2 个账户余额大于0。
[客户端B] 尝试插入一个新账户 (ID 3, balance 200.00)...
[客户端B] *** 期望此处被阻塞，直到客户端A提交。 ***
[客户端B] 尝试插入账户 3 (Charlie, 200.00)...
[客户端B] 成功插入账户 3。
[客户端B] 插入完成，客户端A已提交。
[客户端A] 查询余额大于0的账户数量...
[客户端A] 查询结果：2 个账户余额大于0。
[客户端A] *** 成功防止幻读！第二次查询结果与第一次相同。 ***
[客户端A] 提交事务。

测试场景：防止幻读 (REPEATABLE READ) 结束。
=====
```

图 19: REPEATABLE READ 防止幻读输出

在 REPEATABLE READ 隔离级别下，客户端 A 第一次查询余额大于 0 的账户数量为 2。客户端 B 尝试插入新账户，其 INSERT 操作被阻塞（预期行为）。客户端 A 在第二次查询时，账户数量仍然为 2，没有看到新插入的行。待客户端 A 提交事务后，客户端 B 的插入才得以完成。这清晰地证明了 REPEATABLE READ 隔离级别下幻读被阻止。MySQL InnoDB 通过使用 Next-Key Locks（包括间隙锁）来锁定查询范围内的行及其间隙，从而阻止其他事务在该范围内插入新行。

2.3.5 所有并发问题防范：SERIALIZABLE 隔离级别

SERIALIZABLE 是最高的事务隔离级别，它通过完全串行化事务的执行来避免所有并发问题。

在 SERIALIZABLE 隔离级别下，客户端 A 第一次查询余额大于 0 的账户数量为 2。其查询操作会为查询范围加共享锁。当客户端 B 尝试插入新账户时，其 INSERT 操作被阻塞，直到客户端 A 提交事务并释放锁。客户端 A 的两次查询结果均为 2。这验证了 SERIALIZABLE 隔离级别成功阻止了幻读以及其他所有并发问题。这种高隔离性是以牺牲系统并发性能为代价的，因为引入了更多的锁和更长的锁持有时长，会导致显著的事务阻塞。

```

=====
开始测试场景：所有并发问题防范（SERIALIZABLE）
隔离级别：SERIALIZABLE
=====
数据库和TestBalance表已初始化，并插入初始数据。
[客户端B] 开始事务...
[客户端A] 开始事务...
[客户端A] 查询余额大于0的账户数量...
[客户端A] 查询结果：2 个账户余额大于0。
[客户端B] 尝试插入一个新账户（ID 3，balance 200.00）...
[客户端B] *** 期望此处被阻塞，直到客户端A提交。 ***
[客户端B] 尝试插入账户 3（Charlie，200.00）...
[客户端A] 完成第一次查询和等待。
[客户端A] 第二次查询余额大于0的账户数量（期望与第一次相同）...
[客户端A] 第二次查询结果：2 个账户余额大于0。
[客户端A] *** 成功防止幻读！第二次查询结果与第一次相同。 ***
[客户端A] 提交事务。
[客户端B] 成功插入账户 3。
[客户端B] 插入完成，客户端A已提交。
[客户端B] 提交事务。

测试场景：所有并发问题防范（SERIALIZABLE）结束。
=====

=====
所有测试完成，最终数据状态：
=====
[最终检查] 查询账户 1 余额：1000.00
[最终检查] 查询账户 2 余额：500.00
[最终检查] 查询账户 3 余额：200.00

```

图 20: SERIALIZABLE 隔离级别防止所有并发问题输出

2.4 实验总结与分析

此并发控制实验全面地验证了 MySQL 数据库在不同事务隔离级别下处理并发行为的特点。实验结果清晰地展示了：

- **READ UNCOMMITTED** 级别几乎不提供隔离，导致了脏读、不可重复读和幻读。
- **READ COMMITTED** 级别通过只读取已提交数据，成功阻止了脏读。但它未能阻止不可重复读和幻读，因为事务每次读总是获取最新的已提交版本。
- **REPEATABLE READ** 级别在 MySQL 中表现出强大的隔离能力。它不仅阻止了脏读和不可重复读。此外，MySQL 的 UPDATE 语句默认会加行级排他锁，从而避免了丢失修改问题。
- **SERIALIZABLE** 级别提供了最严格的隔离，确保了所有并发事务的执行如同串行执行一般，彻底杜绝了所有并发问题。但这种强隔离性是以牺牲并发性能为代价的，会引入更多的锁等待。

在实际应用中，选择合适的事务隔离级别是权衡数据一致性要求和系统并发性能的关键。通常的原则是选择能够满足业务一致性要求的最低隔离级别，以最大化系统的并发吞吐量。MySQL 的 REPEATABLE READ 级别因其在一致性和性能之间取得的良好平衡，成为许多应用程序的默认和常用选择。

3 心得体会

本次数据库课程的实验，特别是关于存储过程、触发器以及并发控制的部分，给我带来了深刻的实践体验和理论认知提升。

在存储过程和触发器实验中，我亲手将业务逻辑从前端代码剥离，封装到数据库服务器端的存储过程里。这让我直观地感受到了数据库作为“数据大脑”的强大能力。通过 JDBC 调用这些存储过程，我体会到了前后端协作的效率，以及业务逻辑集中管理所带来的维护便利。触发器的运用更是令人惊喜，它像数据库的“自动守卫”，能够不经人工干预就响应数据变化，这在数据审计和完整性维护方面展现了巨大的价值。尽管在实践中遇到了一些 JDBC 与 MySQL 版本兼容性的“小插曲”，但通过查阅资料和调试，最终解决了问题，这本身就是一次宝贵的学习经历。

并发控制实验更是让我大开眼界。之前仅仅停留在书本上的“脏读”、“不可重复读”、“幻读”和“丢失修改”等概念，通过 Python 模拟两个并发客户端的真实场景，变得具象而生动。我亲眼看到在低隔离级别下数据如何“出错”——事务间彼此影响，数据仿佛在“跳舞”，这让我切实体会到并发问题对数据一致性的巨大威胁。随后，逐级提升隔离级别，观察数据库是如何通过锁、快照等机制巧妙地解决这些问题的，比如 MySQL 在 REPEATABLE READ 下对幻读的独特处理，以及 UPDATE 语句的加锁如何避免了丢失修改。这不仅仅是理论的验证，更让我对数据库系统背后精密的并发控制算法实现充满了敬佩。

总的来说，这次实验不仅仅是完成了作业任务，更是一次深入理解数据库系统“内功”的旅程。它让我认识到，一个健壮可靠的数据库系统，不仅需要高效的存储和查询能力，更离不开严谨的事务管理、精巧的并发控制以及完善的日志备份机制。这些实践经验将为我未来在软件开发和数据管理领域的工作打下坚实的基础。

A 附录 A: 并发实验代码

```
1 import pymysql
2 import time
3 import threading
4 import sys
5 import decimal
6
7 # --- 数据库连接参数 ---
8 DB_CONFIG = {
9     'host': 'localhost',
10    'port': 3306,
11    'user': 'root',
12    'password': '密码',
13    'database': 'airlinedb',
14    'charset': 'utf8mb4',
15    'cursorclass': pymysql.cursors.DictCursor
16 }
17
18 # 用于在多线程中同步打印输出，避免输出混乱
19 print_lock = threading.Lock()
20
21 # --- 辅助函数 ---
22 def create_db_connection(db_config):
23     """创建并返回一个数据库连接，禁用自动提交"""
24     try:
25         conn = pymysql.connect(**db_config)
26         conn.autocommit(False) # 禁用自动提交，手动控制事务
27         return conn
28     except pymysql.MySQLError as e:
29         with print_lock:
30             print(f"数据库连接失败: {e}", file=sys.stderr)
31         return None
32
33 def setup_db_and_table(conn):
34     """设置数据库和 TestBalance 表，并插入初始数据"""
35     try:
36         with conn.cursor() as cursor:
37             # 确保表结构存在，并且每次测试前清空并重新插入初始数据
38             cursor.execute("TRUNCATE TABLE TestBalance;") # 清空表
39             cursor.execute("INSERT INTO TestBalance (id, account_name
```

```

        , balance) VALUES (1, 'Alice', 1000.00);")
40     cursor.execute("INSERT INTO TestBalance (id, account_name
        , balance) VALUES (2, 'Bob', 500.00);")
41     conn.commit()
42     with print_lock:
43         print("数据库和TestBalance表已初始化，并插入初始数据。")
44     return True
45 except pymysql.MySQLError as e:
46     with print_lock:
47         print(f"初始化数据库和表失败:{e}", file=sys.stderr)
48     return False
49
50 def get_balance(conn, account_id, client_name):
51     """查询并打印指定账户的余额"""
52     try:
53         with conn.cursor() as cursor:
54             cursor.execute(f"SELECT balance FROM TestBalance WHERE id
                = {account_id};")
55             result = cursor.fetchone()
56             if result:
57                 with print_lock:
58                     print(f"[{client_name}] 查询账户 {account_id} 余
                        额: {result['balance']:.2f}")
59                     return result['balance']
60             else:
61                 with print_lock:
62                     print(f"[{client_name}] 账户 {account_id} 不存
                        在。")
63                 return None
64     except pymysql.MySQLError as e:
65         with print_lock:
66             print(f"[{client_name}] 查询余额时出错:{e}", file=sys.
                stderr)
67         return None
68
69 def update_balance(conn, account_id, amount, client_name):
70     """更新指定账户的余额"""
71     try:
72         with print_lock:
73             print(f"[{client_name}] 尝试更新账户 {account_id}，金额:
                {'+' if amount > 0 else '-'}{amount:.2f}")

```

```

74         with conn.cursor() as cursor:
75             rows_affected = cursor.execute(f"UPDATE TestBalance SET
              balance=balance+{amount} WHERE id={account_id};"
              )
76         with print_lock:
77             if rows_affected > 0:
78                 print(f"[{client_name}] 成功更新账户 {account_id}。")
79             else:
80                 print(f"[{client_name}] 未能更新账户 {account_id} (可
              能账户不存在或无变化)。")
81         return True
82     except pymysql.MySQLError as e:
83         with print_lock:
84             print(f"[{client_name}] 更新余额时出错: {e}", file=sys.
              stderr)
85         return False
86
87 def insert_account(conn, account_id, account_name, balance,
client_name):
88     """插入一个新账户"""
89     try:
90         with print_lock:
91             print(f"[{client_name}] 尝试插入账户 {account_id} ({
              account_name}, {balance:.2f}) ...")
92         with conn.cursor() as cursor:
93             cursor.execute(f"INSERT INTO TestBalance (id,
              account_name, balance) VALUES ({account_id}, '{
              account_name}', {balance:.2f});")
94         with print_lock:
95             print(f"[{client_name}] 成功插入账户 {account_id}。")
96         return True
97     except pymysql.MySQLError as e:
98         with print_lock:
99             print(f"[{client_name}] 插入账户时出错: {e}", file=sys.
              stderr)
100         return False
101
102 def count_balance_gt_zero(conn, client_name):
103     """查询余额大于0的账户数量"""
104     try:
105         with print_lock:

```



```

106         print(f"[{client_name}]_查询余额大于0的账户数量...")
107     with conn.cursor() as cursor:
108         cursor.execute("SELECT COUNT(*) AS count FROM TestBalance
109             WHERE balance > 0;")
110         result = cursor.fetchone()
111         count = result['count']
112     with print_lock:
113         print(f"[{client_name}]_查询结果: {count} 个账户余额大于0
114             。")
115     return count
116 except pymysql.MySQLError as e:
117     with print_lock:
118         print(f"[{client_name}]_查询账户数量时出错: {e}", file=
119             sys.stderr)
120     return None
121
122 # --- 并发场景测试函数 ---
123
124 def run_test_scenario(test_name, isolation_level_str, client_a_logic,
125     client_b_logic):
126     """
127     运行一个并发测试场景。
128     :param test_name: 场景名称
129     :param isolation_level_str: 隔离级别字符串 (如 'READ UNCOMMITTED
130         ')
131     :param client_a_logic: 客户端A的逻辑函数
132     :param client_b_logic: 客户端B的逻辑函数
133     """
134     with print_lock:
135         print(f"\n{'='*80}\n开始测试场景: {test_name}\n隔离级别: {
136             isolation_level_str}\n{'='*80}")
137
138     conn_a = create_db_connection(DB_CONFIG)
139     conn_b = create_db_connection(DB_CONFIG)
140
141     if not conn_a or not conn_b:
142         with print_lock:
143             print("未能建立所有数据库连接, 跳过此测试。")
144         return
145
146     # 重置数据, 确保每个测试场景从干净的状态开始

```

```

141 if not setup_db_and_table(conn_a): # 只需一个连接重置，因为是
    COMMIT后生效
142     with print_lock:
143         print("数据重置失败，跳过此测试。")
144         conn_a.close()
145         conn_b.close()
146         return
147
148 # 设置每个会话的隔离级别
149 try:
150     with conn_a.cursor() as cursor_a, conn_b.cursor() as cursor_b
        :
151         cursor_a.execute(f"SET SESSION TRANSACTION ISOLATION
            LEVEL {isolation_level_str};")
152         cursor_b.execute(f"SET SESSION TRANSACTION ISOLATION
            LEVEL {isolation_level_str};")
153         conn_a.commit() # 提交隔离级别设置
154         conn_b.commit()
155 except pymysql.MySQLError as e:
156     with print_lock:
157         print(f"设置隔离级别失败: {e}，跳过此测试。", file=sys.
            stderr)
158         conn_a.close()
159         conn_b.close()
160         return
161
162 # 用于线程间同步的事件
163 event_a_ready = threading.Event() # A准备好
164 event_b_ready = threading.Event() # B准备好
165 event_a_signal_b = threading.Event() # A给B信号
166 event_b_signal_a = threading.Event() # B给A信号
167 event_a_done = threading.Event() # A完成
168 event_b_done = threading.Event() # B完成
169
170 # 创建并启动线程
171 thread_a = threading.Thread(target=client_a_logic, args=(conn_a,
    "客户端A",
172                                     event_a_ready, event_b_ready,
                                        event_a_signal_b, event_b_signal_a
                                        , event_a_done, event_b_done))
173 thread_b = threading.Thread(target=client_b_logic, args=(conn_b,

```

```

174         "客户端B",
                                event_b_ready, event_a_ready,
                                event_b_signal_a, event_a_signal_b
                                , event_b_done, event_a_done))

175
176     thread_a.start()
177     thread_b.start()
178
179     thread_a.join() # 等待线程A完成
180     thread_b.join() # 等待线程B完成
181
182     # 关闭连接
183     # conn_a.close()
184     # conn_b.close()
185
186     with print_lock:
187         print(f"\n测试场景:_{test_name}_结束。 \n{' '*80}")
188
189 # --- 客户端逻辑函数（每个并发问题对应一对） ---
190
191 # --- 1. 脏读 (Dirty Read) 演示 ---
192
193 # 场景1.1: 脏读 (READ UNCOMMITTED)
194 def client_a_dirty_read(conn, name, my_ready, other_ready,
195     my_signal_other, other_signal_my, my_done, other_done):
196     try:
197         my_ready.set() # 线程A准备好
198         other_ready.wait() # 等待线程B准备好
199
200         with print_lock: print(f"[{name}]_开始事务...")
201         conn.begin() # START TRANSACTION
202
203         update_balance(conn, 1, -200.00, name) # Alice余额变为800.00
204             , 未提交
205         with print_lock: print(f"[{name}]_更新Alice余额为800.00, 未提
206             交。")
207
208         my_signal_other.set() # 通知客户端B可以读取脏数据了
209         other_signal_my.wait() # 等待客户端B读完脏数据
210
211         with print_lock: print(f"[{name}]_回滚事务, Alice余额将恢复为

```

```

        1000.00。")
209         conn.rollback()
210
211         my_signal_other.set() # 通知客户端B可以再次读取了
212         other_signal_my.wait() # 等待客户端B完成
213
214         my_done.set()
215     except pymysql.MySQLError as e:
216         with print_lock: print(f"[{name}]发生错误:{e}")
217         conn.rollback()
218     finally:
219         if conn.open: conn.close()
220
221 def client_b_dirty_read(conn, name, my_ready, other_ready,
222     my_signal_other, other_signal_my, my_done, other_done):
223     try:
224         my_ready.set()
225         other_ready.wait()
226
227         with print_lock: print(f"[{name}]开始事务...")
228         conn.begin() # START TRANSACTION
229
230         other_signal_my.wait() # 等待客户端A更新并通知
231         get_balance(conn, 1, name) # 第一次读取 Alice 余额, 期望读到
232             脏数据 (800.00)
233
234         my_signal_other.set() # 通知客户端A已读完脏数据
235         other_signal_my.wait() # 等待客户端A回滚并通知
236
237         get_balance(conn, 1, name) # 第二次读取 Alice 余额, 期望读到
238             回滚后的数据 (1000.00)
239
240         with print_lock: print(f"[{name}]提交事务。")
241         conn.commit()
242         my_done.set()
243     except pymysql.MySQLError as e:
244         with print_lock: print(f"[{name}]发生错误:{e}")
245         conn.rollback()
246     finally:
247         if conn.open: conn.close()

```

```

246 # 场景1.2: 防止脏读 (READ COMMITTED)
247 def client_a_prevent_dirty_read(conn, name, my_ready, other_ready,
    my_signal_other, other_signal_my, my_done, other_done):
248     try:
249         my_ready.set()
250         other_ready.wait()
251
252         with print_lock: print(f"[{name}]开始事务...")
253         conn.begin()
254
255         update_balance(conn, 1, -200.00, name) # Alice余额变为800.00
            , 未提交
256         with print_lock: print(f"[{name}]更新Alice余额为800.00, 未提
            交。")
257
258         my_signal_other.set() # 通知客户端B可以读取了
259         other_signal_my.wait() # 等待客户端B读完
260
261         with print_lock: print(f"[{name}]回滚事务, Alice余额将恢复为
            1000.00。")
262         conn.rollback()
263
264         my_signal_other.set() # 通知客户端B可以再次读取了
265         other_signal_my.wait() # 等待客户端B完成
266
267         my_done.set()
268     except pymysql.MySQLError as e:
269         with print_lock: print(f"[{name}]发生错误:{e}")
270         conn.rollback()
271     finally:
272         if conn.open: conn.close()
273
274 def client_b_prevent_dirty_read(conn, name, my_ready, other_ready,
    my_signal_other, other_signal_my, my_done, other_done):
275     try:
276         my_ready.set()
277         other_ready.wait()
278
279         with print_lock: print(f"[{name}]开始事务...")
280         conn.begin()
281

```

```

282     other_signal_my.wait() # 等待客户端A更新并通知
283     get_balance(conn, 1, name) # 第一次读取 Alice 余额，期望读到
        原始值 (1000.00)
284
285     my_signal_other.set() # 通知客户端A已读完
286     other_signal_my.wait() # 等待客户端A回滚并通知
287
288     get_balance(conn, 1, name) # 第二次读取 Alice 余额，期望读到
        原始值 (1000.00)
289
290     with print_lock: print(f"[{name}]提交事务。")
291     conn.commit()
292     my_done.set()
293 except pymysql.MySQLError as e:
294     with print_lock: print(f"[{name}]发生错误:{e}")
295     conn.rollback()
296 finally:
297     if conn.open: conn.close()
298
299 # --- 2. 不可重复读 (Non-repeatable Read) 演示 ---
300
301 # 场景2.1: 不可重复读 (READ COMMITTED)
302 def client_a_non_repeatable_read(conn, name, my_ready, other_ready,
        my_signal_other, other_signal_my, my_done, other_done):
303     try:
304         my_ready.set()
305         other_ready.wait()
306
307         with print_lock: print(f"[{name}]开始事务...")
308         conn.begin()
309
310         get_balance(conn, 1, name) # 第一次读取 Alice 余额 (期望1000
            .00)
311
312         my_signal_other.set() # 通知客户端B可以更新并提交了
313         other_signal_my.wait() # 等待客户端B更新并提交
314
315         get_balance(conn, 1, name) # 第二次读取 Alice 余额，期望读到
            已提交的更新 (900.00)
316
317         with print_lock: print(f"[{name}]提交事务。")

```

```

318         conn.commit()
319         my_done.set()
320     except pymysql.MySQLError as e:
321         with print_lock: print(f"[{name}] 发生错误: {e}")
322         conn.rollback()
323     finally:
324         if conn.open: conn.close()
325
326 def client_b_non_repeatable_read(conn, name, my_ready, other_ready,
327     my_signal_other, other_signal_my, my_done, other_done):
328     try:
329         my_ready.set()
330         other_ready.wait()
331
332         with print_lock: print(f"[{name}] 开始事务...")
333         conn.begin()
334
335         other_signal_my.wait() # 等待客户端A第一次读取
336         update_balance(conn, 1, -100.00, name) # Alice余额变为900.00
337         with print_lock: print(f"[{name}] 提交更新Alice余额到900.00。")
338         conn.commit()
339
340         my_signal_other.set() # 通知客户端A可以进行第二次读取
341         my_done.set()
342     except pymysql.MySQLError as e:
343         with print_lock: print(f"[{name}] 发生错误: {e}")
344         conn.rollback()
345     finally:
346         if conn.open: conn.close()
347
348 # 场景2.2: 防止不可重复读 (REPEATABLE READ)
349 def client_a_prevent_non_repeatable_read(conn, name, my_ready,
350     other_ready, my_signal_other, other_signal_my, my_done, other_done
351 ):
352     try:
353         my_ready.set()
354         other_ready.wait()
355
356         with print_lock: print(f"[{name}] 开始事务...")
357         conn.begin()

```

```

355         get_balance(conn, 1, name) # 第一次读取 Alice 余额 (期望1000
356             .00)
357
358         my_signal_other.set() # 通知客户端B可以更新并提交了
359         other_signal_my.wait() # 等待客户端B更新并提交
360
361         get_balance(conn, 1, name) # 第二次读取 Alice 余额, 期望读到
362             和第一次相同的值 (1000.00)
363
364         with print_lock: print(f"[{name}]_提交事务。")
365         conn.commit()
366         my_done.set()
367     except pymysql.MySQLError as e:
368         with print_lock: print(f"[{name}]_发生错误:_{e}")
369         conn.rollback()
370     finally:
371         if conn.open: conn.close()
372
373 def client_b_prevent_non_repeatable_read(conn, name, my_ready,
374     other_ready, my_signal_other, other_signal_my, my_done, other_done
375 ):
376     try:
377         my_ready.set()
378         other_ready.wait()
379
380         with print_lock: print(f"[{name}]_开始事务...")
381         conn.begin()
382
383         other_signal_my.wait() # 等待客户端A第一次读取
384         update_balance(conn, 1, -100.00, name) # Alice余额变为900.00
385         with print_lock: print(f"[{name}]_提交更新Alice余额到900.00。
386             ")
387         conn.commit()
388
389         my_signal_other.set() # 通知客户端A可以进行第二次读取
390         my_done.set()
391     except pymysql.MySQLError as e:
392         with print_lock: print(f"[{name}]_发生错误:_{e}")
393         conn.rollback()
394     finally:

```



```

391         if conn.open: conn.close()
392
393 # --- 3. 丢失修改 (Lost Update) 演示 ---
394
395 # 场景3.1: 丢失修改 (READ COMMITTED 理论演示, MySQL实际会加锁阻止)
396 # 注意: 在MySQL中, 即使是READ COMMITTED, UPDATE语句也会加行锁, 通常会
      阻止丢失修改。
397 # 这里的演示更多是概念性的, 或者在某些不加锁的数据库/特定场景下可能发
      生。
398 # 在MySQL中, 客户端B的UPDATE会阻塞直到客户端A提交。
399 def client_a_lost_update(conn, name, my_ready, other_ready,
      my_signal_other, other_signal_my, my_done, other_done):
400     try:
401         my_ready.set()
402         other_ready.wait()
403
404         with print_lock: print(f"[{name}]开始事务...")
405         conn.begin()
406
407         balance = get_balance(conn, 1, name) # 读取 Alice 余额
408         # 将 float 数字面量转换为 Decimal 类型
409         amount_to_add = decimal.Decimal('50.00')
410         with print_lock: print(f"[{name}]计算新余额: {balance:.2f} +
      {amount_to_add:.2f} = {balance + amount_to_add:.2f}")
411         time.sleep(0.1) # 模拟计算延迟
412
413         my_signal_other.set() # 通知客户端B可以读取和计算了
414         other_signal_my.wait() # 等待客户端B完成读取和计算
415
416         update_balance(conn, 1, +50.00, name) # 这里参数已经是数值,
      pymysql会处理
417         with print_lock: print(f"[{name}]提交事务。")
418         conn.commit() # 释放锁
419
420         my_done.set()
421     except pymysql.MySQLError as e:
422         with print_lock: print(f"[{name}]发生错误: {e}")
423         conn.rollback()
424     finally:
425         if conn.open: conn.close()
426

```

```

427 def client_b_lost_update(conn, name, my_ready, other_ready,
    my_signal_other, other_signal_my, my_done, other_done):
428     try:
429         my_ready.set()
430         other_ready.wait()
431
432         with print_lock: print(f"[{name}]开始事务...")
433         conn.begin()
434
435         other_signal_my.wait() # 等待客户端A读取和计算
436         balance = get_balance(conn, 1, name) # 读取 Alice 余额
437         # 将 float 数字面量转换为 Decimal 类型
438         amount_to_add = decimal.Decimal('100.00')
439         with print_lock: print(f"[{name}]计算新余额: {balance:.2f} +
            {amount_to_add:.2f} = {balance + amount_to_add:.2f}")
440         time.sleep(0.1) # 模拟计算延迟
441
442         my_signal_other.set() # 通知客户端A已读完和计算完
443
444         with print_lock: print(f"[{name}]尝试更新, 期望会被客户端A阻塞。")
445         update_balance(conn, 1, +100.00, name) # 这里参数已经是数值,
            pymysql会处理
446         with print_lock: print(f"[{name}]提交事务。")
447         conn.commit()
448
449         my_done.set()
450     except pymysql.MySQLError as e:
451         with print_lock: print(f"[{name}]发生错误: {e}")
452         conn.rollback()
453     finally:
454         if conn.open: conn.close()
455
456 # 场景3.2: 防止丢失修改 (REPEATABLE READ)
457 # 在MySQL中, REPEATABLE READ下的UPDATE操作会加行锁, 防止丢失修改。
458 # 这里的测试和上面的3.1在行为上会类似, 因为更新操作在MySQL中默认会加
    锁。
459 # 主要的演示是说明REPEATABLE READ能自然地防止此问题。
460 def client_a_prevent_lost_update_rr(conn, name, my_ready, other_ready,
    my_signal_other, other_signal_my, my_done, other_done):
461     try:

```

```

462     my_ready.set()
463     other_ready.wait()
464
465     with print_lock: print(f"[{name}]_开始事务...")
466     conn.begin()
467
468     get_balance(conn, 1, name) # 读取 Alice 余额
469
470     my_signal_other.set() # 通知客户端B读取
471     other_signal_my.wait() # 等待客户端B读取完成
472
473     update_balance(conn, 1, +50.00, name) # Alice余额变为1050.00
474     , 获得锁
475     with print_lock: print(f"[{name}]_提交事务。")
476     conn.commit() # 释放锁
477
478     my_done.set()
479 except pymysql.MySQLError as e:
480     with print_lock: print(f"[{name}]_发生错误:_{e}")
481     conn.rollback()
482 finally:
483     if conn.open: conn.close()
484
485 def client_b_prevent_lost_update_rr(conn, name, my_ready, other_ready
486 , my_signal_other, other_signal_my, my_done, other_done):
487     try:
488         my_ready.set()
489         other_ready.wait()
490
491         with print_lock: print(f"[{name}]_开始事务...")
492         conn.begin()
493
494         other_signal_my.wait() # 等待客户端A读取
495         get_balance(conn, 1, name) # 读取 Alice 余额
496
497         my_signal_other.set() # 通知客户端A已读完
498
499         with print_lock: print(f"[{name}]_尝试更新, 期望会被客户端A阻
500             塞。")
501         update_balance(conn, 1, +100.00, name) # Alice余额变为1150.00

```

```

500         with print_lock: print(f"[{name}]_提交事务。")
501         conn.commit()
502         my_done.set()
503     except pymysql.MySQLError as e:
504         with print_lock: print(f"[{name}]_发生错误:_{e}")
505         conn.rollback()
506     finally:
507         if conn.open: conn.close()
508
509
510 # --- 4. 幻读 (Phantom Read) 演示 ---
511
512 # 场景4.1: 幻读 (READ COMMITTED)
513 def client_a_phantom_read(conn, name, my_ready, other_ready,
514     my_signal_other, other_signal_my, my_done, other_done):
515     try:
516         my_ready.set()
517         other_ready.wait()
518
519         with print_lock: print(f"[{name}]_开始事务...")
520         conn.begin()
521
522         count1 = count_balance_gt_zero(conn, name) # 第一次查询账户数
523             量
524
525         my_signal_other.set() # 通知客户端B可以插入了
526         other_signal_my.wait() # 等待客户端B插入并提交
527
528         count2 = count_balance_gt_zero(conn, name) # 第二次查询账户数
529             量, 期望看到新插入的行
530
531         if count2 > count1:
532             with print_lock: print(f"[{name}]_***_发生了幻读! 第二次
533                 查询看到了新插入的行。_***")
534         else:
535             with print_lock: print(f"[{name}]_没有发生幻读_(不符合
536                 READ_COMMITTED预期)。")
537
538         with print_lock: print(f"[{name}]_提交事务。")
539         conn.commit()
540         my_done.set()

```

```

536     except pymysql.MySQLError as e:
537         with print_lock: print(f"[{name}] 发生错误: {e}")
538         conn.rollback()
539     finally:
540         if conn.open: conn.close()
541
542 def client_b_phantom_read(conn, name, my_ready, other_ready,
543     my_signal_other, other_signal_my, my_done, other_done):
544     try:
545         my_ready.set()
546         other_ready.wait()
547
548         with print_lock: print(f"[{name}] 开始事务...")
549         conn.begin()
550
551         other_signal_my.wait() # 等待客户端A第一次查询
552
553         insert_account(conn, 3, 'Charlie', 200.00, name) # 插入新账户
554         with print_lock: print(f"[{name}] 提交新账户插入。")
555         conn.commit() # 提交新插入的行
556
557         my_signal_other.set() # 通知客户端A可以进行第二次查询
558         my_done.set()
559     except pymysql.MySQLError as e:
560         with print_lock: print(f"[{name}] 发生错误: {e}")
561         conn.rollback()
562     finally:
563         if conn.open: conn.close()
564
565 # 场景4.2: 防止幻读 (REPEATABLE READ)
566 def client_a_prevent_phantom_read_rr(conn, name, my_ready,
567     other_ready, my_signal_other, other_signal_my, my_done, other_done
568 ):
569     try:
570         my_ready.set()
571         other_ready.wait()
572
573         with print_lock: print(f"[{name}] 开始事务...")
574         conn.begin()
575
576         count1 = count_balance_gt_zero(conn, name) # 第一次查询账户数

```

量

```
574
575 my_signal_other.set() # 通知客户端B可以插入了（期望B被阻塞）
576 other_signal_my.wait() # 等待客户端B尝试插入（期望B被阻塞，直到A提交）
577
578 count2 = count_balance_gt_zero(conn, name) # 第二次查询账户数量，期望与第一次相同
579
580 if count2 == count1:
581     with print_lock: print(f"[{name}] *** 成功防止幻读！第二次查询结果与第一次相同。 ***")
582 else:
583     with print_lock: print(f"[{name}] 幻读发生（不符合REPEATABLE READ预期）。")
584
585 with print_lock: print(f"[{name}] 提交事务。")
586 conn.commit() # 释放锁
587 my_done.set() # 通知客户端B可以继续了（如果它被阻塞）
588 except pymysql.MySQLError as e:
589     with print_lock: print(f"[{name}] 发生错误：{e}")
590     conn.rollback()
591 finally:
592     if conn.open: conn.close()
593
594 def client_b_prevent_phantom_read_rr(conn, name, my_ready,
595     other_ready, my_signal_other, other_signal_my, my_done, other_done):
596     try:
597         my_ready.set()
598         other_ready.wait()
599
600         with print_lock: print(f"[{name}] 开始事务...")
601         conn.begin()
602
603         other_signal_my.wait() # 等待客户端A第一次查询并加锁
604
605         with print_lock: print(f"[{name}] 尝试插入一个新账户（ID 3, balance 200.00）...")
606         with print_lock: print(f"[{name}] *** 期望此处被阻塞，直到客户端A提交。 ***")
```

```

606         insert_account(conn, 3, 'Charlie', 200.00, name) # 插入新账户
           (期望会被阻塞)
607
608         with print_lock: print(f"[{name}]_插入完成, 客户端A已提交。")
609         conn.commit() # 提交新插入的行
610
611         my_signal_other.set() # 通知客户端A已完成
612         my_done.set()
613     except pymysql.MySQLError as e:
614         with print_lock: print(f"[{name}]_发生错误:_{e}")
615         conn.rollback()
616     finally:
617         if conn.open: conn.close()
618
619 # --- 5. 完全串行化 (SERIALIZABLE) 演示 ---
620 # SERIALIZABLE 级别会防止所有并发问题。我通过幻读场景来演示其效果。
621 # 修改 client_a_serializable 函数
622 def client_a_serializable(conn, name, my_ready, other_ready,
        my_signal_other, other_signal_my, my_done, other_done):
623     try:
624         my_ready.set()
625         other_ready.wait()
626
627         with print_lock: print(f"[{name}]_开始事务...")
628         conn.begin()
629
630         count1 = count_balance_gt_zero(conn, name) # 第一次查询账户数
           量 (SERIALIZABLE将加范围锁)
631
632         my_signal_other.set() # 通知客户端B可以插入了 (期望会被阻塞)
633         other_signal_my.wait(timeout=30) # 等待客户端B尝试插入 (期望B
           被阻塞, 直到A提交)。设置超时, 避免死锁 (如果逻辑有误)
634         # **这里只是等待B的信号, 实
           际阻塞发生在B的 INSERT语句
           处**
635         # (可以考虑移除这句, 让B一直
           阻塞直到A提交)
636
637         with print_lock: print(f"[{name}]_完成第一次查询和等待。")
638         with print_lock: print(f"[{name}]_第二次查询余额大于0的账户数
           量_(期望与第一次相同)...")

```

```

639         with conn.cursor() as cursor:
640             cursor.execute("SELECT COUNT(*) AS count FROM TestBalance
                               WHERE balance > 0;")
641             result = cursor.fetchone()
642             count2 = result['count']
643         with print_lock: print(f"[{name}] 第二次查询结果: {count2} 个
                               账户余额大于0。")
644
645         if count2 == count1:
646             with print_lock: print(f"[{name}] *** 成功防止幻读! 第二
                                   次查询结果与第一次相同。 ***")
647         else:
648             with print_lock: print(f"[{name}] 幻读发生 (不符合
                                   SERIALIZABLE 预期)。")
649
650         with print_lock: print(f"[{name}] 提交事务。")
651         conn.commit() # 释放锁
652         my_signal_other.set() # 释放锁后, 通知客户端B可以继续了
653
654         # 等待客户端B真正完成, 避免主线程过早关闭连接
655         other_done.wait()
656         my_done.set() # A线程完成
657     except pymysql.MySQLError as e:
658         with print_lock: print(f"[{name}] 发生错误: {e}")
659         conn.rollback()
660     finally:
661         if conn.open: conn.close()
662
663 # 客户端B的逻辑不需要太多修改, 它会在INSERT处自动阻塞, 直到客户端A提
    交。
664 # client_b_serializable 的 my_signal_other.set() 应该在
        insert_account 后执行, 表示它完成了插入操作的尝试。
665 # 确保 client_b_serializable 在 conn.commit() 之后也 set its my_done
        event.
666
667 # 请确保 client_b_serializable 的 my_signal_other.set() 在它尝试
        insert 之后执行,
668 # 并且 client_b_serializable 最后也要 set its my_done event.
669
670 # 检查 client_b_serializable 函数:
671 def client_b_serializable(conn, name, my_ready, other_ready,

```



```

my_signal_other, other_signal_my, my_done, other_done):
    try:
        my_ready.set()
        other_ready.wait()

        with print_lock: print(f"[{name}]_开始事务...")
        conn.begin()

        other_signal_my.wait() # 等待客户端A第一次查询并加锁

        with print_lock: print(f"[{name}]_尝试插入一个新账户(ID_3,
            balance_200.00)...")
        with print_lock: print(f"[{name}]_***_期望此处被阻塞, 直到客
            户端A提交。_***")
        insert_account(conn, 3, 'Charlie', 200.00, name) # 插入新账户
            (期望会被阻塞)

        my_signal_other.set() # **客户端B通知A, 它已经尝试插入了(或
            插入完成了) **

        with print_lock: print(f"[{name}]_插入完成, 客户端A已提交。")
            # 这句将在A提交后才打印, 因为B被阻塞了
        with print_lock: print(f"[{name}]_提交事务。")
        conn.commit() # 提交新插入的行

        my_done.set() # 客户端B完成
    except pymysql.MySQLError as e:
        with print_lock: print(f"[{name}]_发生错误:_{e}")
        conn.rollback()
    finally:
        if conn.open: conn.close()

# --- 主程序逻辑 ---
if __name__ == "__main__":
    LOG_FILE_NAME = "concurrency_experiment_output.txt" # 定义日志文
        件名

    # 保存原始的标准输出流
    original_stdout = sys.stdout

    try:

```

```

706     # 打开日志文件，以写入模式 ('w') 打开，如果文件不存在则创建，
       如果存在则清空
707     with open(LOG_FILE_NAME, 'w', encoding='utf-8') as f_log:
708         # 将标准输出流重定向到文件对象
709         sys.stdout = f_log
710
711     # 现在，所有的 print() 输出都会写入到 f_log 文件中
712
713     main_conn = create_db_connection(DB_CONFIG)
714     if not main_conn:
715         # 错误信息通过 sys.stderr 输出，不受 sys.stdout 重定
           向影响
716         # 但为了确保错误也能被记录到文件，需要手动捕获
717         with original_stdout: # 临时切换回原始 stdout 打印错
           误，然后恢复
718             print(f"Error: 无法建立主数据库连接，实验终止。",
                   , file=sys.stderr)
719         sys.exit(1)
720
721     if not setup_db_and_table(main_conn):
722         with original_stdout:
723             print(f"Error: 初始化数据库和表失败，实验终止。",
                   , file=sys.stderr)
724             main_conn.close()
725             sys.exit(1)
726
727     main_conn.close() # 主连接只用于初始化，之后每个线程创建
           自己的连接
728
729     # 所有测试场景
730     scenarios = [
731         # 脏读 (Dirty Read)
732         ("脏读 (Dirty Read) 演示", 'READ_UNCOMMITTED',
          client_a_dirty_read, client_b_dirty_read),
733         ("防止脏读", 'READ_COMMITTED',
          client_a_prevent_dirty_read,
          client_b_prevent_dirty_read),
734         ("防止脏读", 'REPEATABLE_READ',
          client_a_prevent_dirty_read,
          client_b_prevent_dirty_read),
735

```

```

736 # 不可重复读 (Non-repeatable Read)
737 ("不可重复读□(Non-repeatable□Read)□演示", 'READ□
    UNCOMMITTED', client_a_non_repeatable_read,
    client_b_non_repeatable_read),
738 ("不可重复读□(Non-repeatable□Read)□演示", 'READ□
    COMMITTED', client_a_non_repeatable_read,
    client_b_non_repeatable_read),
739 ("防止不可重复读", 'REPEATABLE□READ',
    client_a_prevent_non_repeatable_read,
    client_b_prevent_non_repeatable_read),
740
741 # 丢失修改 (Lost Update)
742 ("丢失修改□(Lost□Update)□演示□(MySQL中会阻塞)", 'READ
    □UNCOMMITTED', client_a_lost_update,
    client_b_lost_update),
743 ("丢失修改□(Lost□Update)□演示□(MySQL中会阻塞)", 'READ
    □COMMITTED', client_a_lost_update,
    client_b_lost_update),
744 ("防止丢失修改□(REPEATABLE□READ)", 'REPEATABLE□READ',
    client_a_prevent_lost_update_rr,
    client_b_prevent_lost_update_rr),
745
746 # 幻读 (Phantom Read)
747 ("幻读□(Phantom□Read)□演示", 'READ□UNCOMMITTED',
    client_a_phantom_read, client_b_phantom_read),
748 ("幻读□(Phantom□Read)□演示", 'READ□COMMITTED',
    client_a_phantom_read, client_b_phantom_read),
749 ("防止幻读□(REPEATABLE□READ)", 'REPEATABLE□READ',
    client_a_prevent_phantom_read_rr,
    client_b_prevent_phantom_read_rr),
750
751 # 完全串行化 (SERIALIZABLE)
752 ("所有并发问题防范□(SERIALIZABLE)", 'SERIALIZABLE',
    client_a_serializable, client_b_serializable)
753 ]
754
755 for name, iso_level, client_a_logic, client_b_logic in
    scenarios:
756     run_test_scenario(name, iso_level, client_a_logic,
    client_b_logic)
757

```

```

758         # 最终验证数据
759         final_conn = create_db_connection(DB_CONFIG)
760         if final_conn:
761             with print_lock: # 使用 print_lock 确保线程安全地打印
762                 print(f"\n{'='*80}\n所有测试完成，最终数据状态:\n
{'='*80}")
763                 get_balance(final_conn, 1, "最终检查")
764                 get_balance(final_conn, 2, "最终检查")
765                 get_balance(final_conn, 3, "最终检查") # 检查是否有
Charlie 账户（根据最后一次测试的提交情况）
766                 final_conn.close()
767
768             with print_lock:
769                 print("\n所有实验已完成。")
770
771     except Exception as e:
772         # 如果在文件操作或主逻辑中发生任何其他异常，确保原始标准输出
被恢复
773         sys.stdout = original_stdout
774         print(f"程序执行过程中发生未预期的错误:_{e}", file=sys.stderr
)
775         sys.exit(1)
776     finally:
777         # 无论如何，确保标准输出被恢复到终端
778         sys.stdout = original_stdout
779         print(f"\n实验输出已保存到:_{LOG_FILE_NAME}") # 这条信息会显
示在终端

```